LoManLe — Local Manifold Learning

Asis Hallab

August 5, 2026

1 Introduction

1.1 Preamble

Document status (2026-08-05). This file is the mathematical design document for LoManLe. It has been cross-checked against the current implementation in `src/lomanle.F90` (last substantially revised 2026-07-27 for performance, and 2026-08-04/05 for the MST Tier-2 complexity fix described below). Every phase below now carries an **Implementation** block naming the actual subroutines and their time/memory complexity, and a **Lab notes** block bringing over what `misc/smoothing_experiments.md` ("the lab book") records as learned or still-open for that phase. Wherever the code does something structurally different from what this document originally prescribed, that is called out explicitly as an **Open experiment** rather than silently glossed over — the design intent is preserved as the thing being tested against, not silently replaced.

For the full narrative (including three rounds of debugging the backbone construction, the complete performance-optimization log, and the current "Open Questions" tracker), see `misc/smoothing_experiments.md`. This document stays the authoritative *mathematical* description; that one is the authoritative *lab book*.

1.2 Objective

Given a noisy finite point cloud

$$X = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}, \qquad \mathbf{x}_i \in \mathbb{R}^D,$$

LoManLe estimates a low-dimensional manifold

$$\mathcal{M} \subset \mathbb{R}^D, \qquad d \ll D,$$

where D is the ambient dimension and d is the intrinsic manifold dimension.

The central approximation is that a sufficiently small neighborhood of a smooth manifold can be represented by its local tangent space. LoManLe estimates these tangent spaces from noisy observations using local covariance eigendecomposition / singular value decomposition (SVD), constructs overlapping local charts, projects observations onto them, stitches compatible charts, and optionally iteratively refines the resulting manifold.

Conceptually,

adaptive neighborhoods → local tangent models → overlapping atlas → projection → stitching → refinement

The local tangent model is the first-order approximation of an underlying smooth manifold.

1.3 Input and output

1.3.1 Input

A matrix

$$X \in \mathbb{R}^{D \times n},$$

with one column $\mathbf{x}_i$ per observation.

Required parameters:

- $k_{\min}$: minimum neighborhood size.
- $k_{\max}$: maximum allowed neighborhood size, potentially $n/4$.
- $g > 1$: geometric neighborhood growth factor, currently approximately 1.25.
- d: desired intrinsic manifold dimension, unless inferred locally.
- overlap requirement for atlas construction.
- numerical criteria for stable local tangent estimation.
- maximum number of refinement iterations.
- convergence/stopping criterion for iterative refinement.

Optional:

- infer d locally from the singular/eigenvalue spectrum;
- smoothing of adaptive kernel bandwidths;
- noise-aware stitching;
- bifurcation/multifurcation detection.

Open experiment: all four optional items above are still unimplemented in `src/lomanle.F90`. d (`manifold_dim`) is always a fixed, caller-supplied scalar; there is no bandwidth-field smoothing pass; stitching is not noise-aware (see Phase X below); and bifurcation/multifurcation status is currently read off *after the fact* from anchor degree in the backbone MST (Phase XII), not detected geometrically during stitching. Each is discussed at its corresponding phase below.

1.3.2 Output

At minimum:

1. projected manifold points

$$\hat{X} = \{\hat{\mathbf{x}}_1, \ldots, \hat{\mathbf{x}}_n\};$$

2. atlas anchors and their memberships;
3. local tangent bases;
4. local intrinsic dimensions if inferred;
5. atlas intersection/connectivity graph;
6. explicit connectivity of the learned manifold where available:
   - edges for $d = 1$;
   - higher-dimensional mesh/simplicial representation for $d > 1$, once implemented;
7. optionally local residual/noise models and reconstruction diagnostics.

1.3.2.1 Implementation

Functions: `lomanle_compute_alloc` (public entry point, allocates all work buffers) -> `lomanle_compute` (owns the outer convergence loop) -> `lomanle_pass` (one full atlas-build-and-stitch pass), all in `src/lomanle.F90`.

Status: all seven output items are actually produced. Item 4 (local intrinsic dimensions) is vacuous since d is fixed, not inferred (see §7). Item 6 is produced only for $d = 1$; the $d > 1$ mesh/simplicial case remains unimplemented (§17, §Phase XII below).

Lab notes. `misc/smoothing_experiments.md` documents the API as still wider than a finished module's would be: `lomanle_compute_alloc` currently returns many intermediate/diagnostic arrays (`densities`, `gap_values`, `k_selected`, `stability_values`, `growth_stopped_complex`, both an iteration-1 and a final snapshot of nearly every quantity, and more) well beyond a "just give me the skeleton" interface. This is intentional while the algorithm design is still being explored — every diagnostic plot the lab book describes depends on one of these fields being exposed — but the interface is explicitly flagged there as needing to be trimmed down once the design settles. **Open experiment:** decide the final, trimmed public API once Phases IX/X and Tangent compatibility (§15, noise-aware stitching) and the $d > 1$ mesh (Phase XII) either land or are abandoned.

1.4 Mathematical foundation

Assume locally that an unknown d-dimensional manifold $\mathcal{M}$ can be parameterized by

$$f : \mathbb{R}^d \to \mathbb{R}^D.$$

Around intrinsic coordinate $\mathbf{u}_0$,

$$f(\mathbf{u}_0 + \Delta\mathbf{u}) = f(\mathbf{u}_0) + J_f(\mathbf{u}_0)\Delta\mathbf{u} + O(\|\Delta\mathbf{u}\|^2).$$

Therefore, locally,

$$\mathcal{M} \approx \mathbf{c} + \text{span}(U_d),$$

where $\mathbf{c}$ is a local center and

$$U_d = [\mathbf{u}_1, \ldots, \mathbf{u}_d]$$

contains an orthonormal basis for the estimated tangent space.

LoManLe estimates U_d from the dominant local variance directions.

Thus each chart is a **data-derived first-order local approximation** of the manifold.

1.5 Implementation status at a glance

A quick-reference map from design phase to code. "Time" gives the dominant asymptotic cost of that step within one `lomanle_pass` call (i.e. one outer iteration); n = `n_points`, n_a = `n_anchor` (number of atlas anchors, typically a small fraction of n — "hundreds of anchors for tens of thousands of points is normal"), $\bar{k}$ = a point's own converged adaptive neighborhood size, T = OpenMP thread count. `lomanle_compute` multiplies all of this by the number of outer iterations actually run ($\leq$ `max_iterations`).

1.5.1 Phase I – Spatial index

1.5.1.1 Step
Step 0

1.5.1.2 Functions
`build_kd_index` (kd_tree mod)

1.5.1.3 Time
$O(n \log n)$

1.5.1.4 Memory
$O(n)$

1.5.1.5 Status
matches

1.5.2 Phases II/III – Adaptive growth

1.5.2.1 Step
Steps 1-3

1.5.2.2 Functions `grow_adaptive_neighborhoods`, `grow_one_point_neighborhood`

1.5.2.3 Time $O(n \log n + n\bar{k})$

1.5.2.4 Memory $O(n) + O(nT)$ scratch

1.5.2.5 Status matches, rule itself was redesigned (see phase section below)

1.5.3 Local intrinsic dimensionality estimation (§7)

1.5.3.1 Step –

1.5.3.2 Functions *(none – not implemented)*

1.5.3.3 Time –

1.5.3.4 Memory –

1.5.3.5 Status **open experiment**

1.5.4 Phase IV – Density/reliability labels

1.5.4.1 Step (inline in Steps 1-3)

1.5.4.2 Functions *(no separate subroutine)*

1.5.4.3 Time included above

1.5.4.4 Memory included above

1.5.4.5 Status matches formula, no bandwidth smoothing

1.5.5 Phase V – Greedy atlas construction

1.5.5.1 Step Steps 4, 5, 5b

1.5.5.2 Functions `construct_atlas` -> `sort_points_by_density`, `select_atlas_anchors`, `absorb_orphans`

1.5.5.3 Time $O(n_a \, n \log n)$

1.5.5.4 Memory $O(n) + O(nT)$ scratch

1.5.5.5 Status matches; dominant unresolved cost (see Performance lab notes below)

1.5.6 Phase VI – Recompute chart geometry

1.5.6.1 Step Step 6

1.5.6.2 Functions `compute_anchor_svd`

1.5.6.3 Time $O(n_a \log n)$

1.5.6.4 Memory $O(n) + O(nT)$ scratch

1.5.6.5 Status matches

1.5.7 Phase VII – Project onto chart

1.5.7.1 Step (fused into Step 10)

1.5.7.2 Functions `stitch_multi_anchor_point`, `stitch_single_anchor_point`

1.5.7.3 Time included in Step 10

1.5.7.4 Memory included in Step 10

1.5.7.5 Status fused, not a separate materialized step

1.5.8 Phase VIII – Chart-overlap graph

1.5.8.1 Step Steps 6.5-9

1.5.8.2 Functions `build_membership_matrix`, `count_anchor_intersection_edges`, `build_point_anchor_lists`, `fill_anchor_intersection_edges`, `build_intersection_graph_alloc/build_intersection_graph`

1.5.8.3 Time $O(n_a\, n)$

1.5.8.4 Memory $O(n\, n_a)$ **dense mask**

1.5.8.5 Status matches structurally; **its BFS output is not used for connectivity decisions** (open experiment)

1.5.9 Phase IX – Normal/noise structure ($\Sigma_\perp$)

1.5.9.1 Step –

1.5.9.2 Functions *(none – only scalar `normal_errors`)*

1.5.9.3 Time –

1.5.9.4 Memory –

1.5.9.5 Status **open experiment** (this session's design discussion)

1.5.10 Phase X – Noise-tube stitch decision

1.5.10.1 Step –

1.5.10.2 Functions *(none – unconditional if `anchor_count>=2`)*

1.5.10.3 Time –

1.5.10.4 Memory –

1.5.10.5 Status **open experiment**, see "Potential Stitching Experiment" below

1.5.11 Tangent compatibility (§15)

1.5.11.1 Step –

1.5.11.2 Functions *(none between different anchors)*

1.5.11.3 Time –

1.5.11.4 Memory –

1.5.11.5 Status **open experiment**; furcation instead falls out structurally from MST degree (Phase XII)

1.5.12 Phase XI – Stitch projections

1.5.12.1 Step Step 10

1.5.12.2 Functions `stitch_points`, `stitch_multi_anchor_point`, `stitch_single_anchor_point`

1.5.12.3 Time $O(n)$

1.5.12.4 Memory $O(n)$

1.5.12.5 Status matches, weight formula simplified to per-anchor (not per point-anchor pair)

1.5.13 Phase XII – Explicit topology ($d = 1$)

1.5.13.1 Step "Step 11" (lab book)

1.5.13.2 Functions `build_skeleton_edges_alloc` -> `build_anchor_mapping`, `build_anchor_mst_alloc` (-> `mark_tier1_candidate_pairs`, `count_tier2_candidate_pairs`, `build_anchor_mst` -> `build_tier1_mst_edges`, `build_tier2_mst_edges`, `classify_anchor_roles`), `build_member_chains`, `build_branch_adjacency`, `build_skeleton_edges` (-> `emit_branch`, `nearest_member`, `find_root`)

1.5.13.3 Time $O(n_a \log n_a) + O(n \log(n/n_a))$

1.5.13.4 Memory $O(n_a^2)$ dense (small) + $O(n)$

1.5.13.5 Status **entirely different mechanism than this doc originally described** – anchor-graph MST, not point-level nearest-neighbor chaining

1.5.14 Phase XII – Mesh for $d > 1$

1.5.14.1 Step –

1.5.14.2 Functions *(none)*

1.5.14.3 Time –

1.5.14.4 Memory –

1.5.14.5 Status **open, not started**

1.5.15 Phase XIII – Iterative refinement

1.5.15.1 Step `lomanle_compute`'s outer loop

1.5.15.2 Functions `lomanle_compute` re-calling `lomanle_pass`

1.5.15.3 Time (all of the above) x iterations

1.5.15.4 Memory $O(n)$ extra for iteration-1 snapshot

1.5.15.5 Status F is literally "rebuild the atlas and restitch," not an external ManLe/ANWIL call

1.5.16 Stop-before-oversmoothing objective (§19)

1.5.16.1 Step –

1.5.16.2 Functions *(none – only `max_disp < conv_tol`)*

1.5.16.3 Time –

1.5.16.4 Memory –

1.5.16.5 Status open experiment; the weighted-objective idea is documented for ManLe/AManLe, not yet ported to LoManLe's own loop

Each phase above is expanded with its full reasoning, functions, and lab notes in the phase sections below.

1.6 Phase I — Spatial index

Construct a k-d tree or another nearest-neighbor index over X.

It provides

$$\mathrm{kNN}(\mathbf{x}_i, k)$$

for arbitrary k.

The index is used for adaptive neighborhood construction, density estimation and local geometric operations.

In sufficiently high ambient dimension, the nearest-neighbor implementation itself may need replacement because k-d trees deteriorate with increasing D.

1.6.0.1 Implementation Functions: `build_kd_index` (module `kd_tree`, `src/k_d_tree.F90`), called once per `lomanle_pass` inside `grow_adaptive_neighborhoods` (`src/lomanle.F90`).

Complexity: time $O(n \log n)$; memory $O(n)$ (index/workspace/value-buffer/permutation/left-stack/right-stack arrays sized `n_points`, plus a `(3, n_points)` recursion stack).

Lab notes. `misc/smoothing_experiments.md` §3 notes this step is unchanged by the 2026-07-27 performance work — it was already the efficient part. The document's own caveat about high-D deterioration (this doc's last sentence above) is echoed directly in the lab book's Principal Limitations section (§22 below): "Euclidean nearest-neighbor relationships may deteriorate, and accumulated noise across many normal dimensions can obscure tangent structure." **Open experiment:** no alternative high-D index (e.g. a ball tree, or an approximate-NN structure) has been implemented or evaluated; this remains a stated but untested limitation.

1.7 Phase II — Adaptive neighborhood estimation

A fixed k is undesirable because sampling density, curvature and noise vary across the manifold.

For every observation $\mathbf{x}_i$, begin with

$$k_i = k_{\min}.$$

Obtain its neighborhood

$$N_i(k_i) = \mathrm{kNN}(\mathbf{x}_i, k_i).$$

Compute the neighborhood center

$$\boldsymbol{\mu}_i = \frac{1}{|N_i|} \sum_{j \in N_i} \mathbf{x}_j.$$

Construct the centered matrix

$$Y_i = [\mathbf{x}_{j_1} - \boldsymbol{\mu}_i, \ldots, \mathbf{x}_{j_k} - \boldsymbol{\mu}_i].$$

Equivalently compute the local covariance matrix

$$C_i = \frac{1}{k_i - 1} Y_i Y_i^\top.$$

Its eigendecomposition is

$$C_i = U_i \Lambda_i U_i^\top,$$

with

$$\lambda_{i,1} \geq \lambda_{i,2} \geq \cdots \geq \lambda_{i,D} \geq 0.$$

This is equivalent, for the required purpose, to obtaining the local principal directions through SVD of Y_i.

1.8 Phase III — Grow neighborhoods until tangent estimation is stable

For each $\mathbf{x}_i$, repeatedly increase the neighborhood geometrically:

$$k_i^{(t+1)} = \min\left(\left\lceil g k_i^{(t)} \right\rceil, k_{\max}\right).$$

At every scale, estimate the local eigensystem.

The neighborhood is accepted at the **smallest scale** at which the local manifold estimate is sufficiently stable.

This early stopping is essential: increasing k reduces variance but eventually introduces curvature bias and may mix nearby branches.

Hence:

choose the smallest neighborhood giving a stable tangent estimate

rather than the largest statistically convenient neighborhood.

1.8.1 Tangent stability

For fixed intrinsic dimension d, compare tangent spaces between consecutive neighborhood sizes.

Let

$$U_d^{(t)}, U_d^{(t+1)}$$

be the two tangent bases.

Use their principal angles

$$0 \leq \theta_1 \leq \cdots \leq \theta_d \leq \frac{\pi}{2}.$$

For example, require

$$\theta_{\max} = \max_j \theta_j \leq \tau_\theta$$

for one or more consecutive growth steps.

The criterion should operate on the **subspace**, not individual eigenvectors, because rotations within a d-dimensional tangent space represent the same tangent space.

1.8.2 Spectral separation

Additionally require evidence that tangent variance is distinguishable from normal variance, for example through

$$G_i(d) = \frac{\lambda_{i,d}}{\lambda_{i,d+1} + \epsilon}.$$

Require

$$G_i(d) \geq \tau_G.$$

The precise stability criterion and thresholds are algorithm parameters and must be reported.

If no acceptable neighborhood is found before $k_{\max}$, flag that location as geometrically unresolved rather than silently treating the largest neighborhood as reliable.

1.8.2.1 Implementation (Phases II + III together) Phases II and III are implemented as a single unit — the code does not compute one fixed-k neighborhood and then separately grow it; each candidate size is scored and the best-so-far is kept as growth proceeds.

Functions: `grow_adaptive_neighborhoods` (`src/lomanle.F90`) is the `!$omp parallel do` driver, one call to `grow_one_point_neighborhood` per point. Uses `kd_knn_query` (kd_tree mod), `sort_array` (f42_utils mod), and `dsyev` (LAPACK, declared `pure` purely as thread-safety documentation).

Complexity: time $O(n \log n + n\bar{k})$, where $\bar{k}$ is a point's own converged neighborhood size (on the order of $k_{\min}$ unless growth needed several steps). Memory: $O(n)$ for the output arrays, plus $O(n)$ **of private automatic-array scratch per active OpenMP thread** inside the parallel region (`n_loc_i`, `d_loc_i`, `workspace_i`, `val_buf_i`, `perm_i`, `l_stack_i`, `r_stack_i` are each sized `n_points`, not $\bar{k}$) — i.e. $O(nT)$ total scratch for T threads. This per-thread sizing is a genuine, currently-unaddressed memory cost that scales with thread count; it has not been flagged in the lab book before now.

Status vs. this document — the growth-acceptance rule was substantially redesigned. The math above (grow geometrically, accept at the smallest scale where principal-angle stability + spectral separation both hold) is still the *intent*, but the *acceptance rule that actually decides which candidate to keep* is different from a literal reading of this section, for reasons the lab book records concretely.

Lab notes (`misc/smoothing_experiments.md` §4). What was wrong with "grow until the gap is big enough" (a literal implementation of §6 above):

- A bad spectral gap $G_i(d)$ was always interpreted as "add more points," but a bad gap can also mean the neighborhood just crossed into a different branch, an intersection, or a region of high curvature — cases where growing further makes the estimate *worse*. The gap alone could not distinguish these.
- The k-d tree query included the point itself (at distance 0) and returned neighbors unsorted, making naive radius/jump comparisons across growth steps meaningless until corrected.
- If the gap never crossed threshold before $k_{\max}$, the code accepted whatever neighborhood it had grown to *last* — precisely the neighborhood most likely to have already crossed into a different branch, not the best one evaluated.

The current rule instead computes a **quality score** at every candidate size,

$$\text{quality} = \min(\text{gap}/g_{\text{threshold}}, 2) + \text{stability} - \text{normal\_error}/\text{local\_scale}^2 - \text{radius}/\text{local\_scale},$$

and keeps the **best-scoring candidate seen so far**, not the last one evaluated. Growth stops once stability drops below `stability_threshold` for `patience` consecutive steps — where `patience` is itself derived per point from a self-calibrating noise ratio ($\sqrt{\text{normal\_error}}/\text{local\_scale}$), not a fixed constant: clean, curve-like data reacts on the first bad reading (`patience=1`); noisy, blob-like data gets more room for a small-sample tangent estimate to settle before a bad reading is trusted. A `scale_factor` safety cap also stops growth outright if the radius has grown far beyond the point's own local scale.

Performance lab notes (`misc/smoothing_experiments.md` §18.6). Profiling revealed Steps 0-3 do *not* always dominate — which of Steps 1-3 ("growth") or Step 5 ("atlas") dominates depends almost entirely on $k_{\min}$ relative to n:

n	$k_{\min}$	growth % of total	atlas % of total
5,000	200	43%	55%
5,000	800	77%	22%
50,000	2,000	45%	55%
50,000	8,000	83%	17%

Within Steps 1-3, `kd_knn_query`+sort is consistently 83–86% of the time regardless of scale (covariance/`dsyev` is only 14–17%). A caching fix was implemented (only re-query the k-d tree when the growth target exceeds what is already cached, fetching a geometrically larger batch each time and serving intermediate steps from a sorted prefix — exact, not approximate) and is bit-identical to the uncached version, but its measured impact on large-$k_{\min}$ cases was negligible: growth doesn't take enough steps in that regime for caching to matter. **Open experiment** (`misc/smoothing_experiments.md` §18.7, still unresolved): a single large-k `kd_knn_query` call is inherently expensive once k approaches a significant fraction of n — a k-d tree prunes poorly once the query radius already covers a large slice of the dataset — and no low-risk fix is known yet. Structurally the same phenomenon as Step 5's radius-query pruning degradation (§18.2, Phase V below).

1.9 Optional local intrinsic dimensionality estimation

Instead of prescribing d, inspect

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D.$$

Candidate dimensionalities correspond to stable spectral separations such as

$$G(r) = \frac{\lambda_r}{\lambda_{r+1} + \epsilon}.$$

Select a dimension d_i only if the inferred dimension and associated tangent subspace remain stable while the neighborhood grows.

Thus different charts may have

$$d_i \neq d_j.$$

This permits locally changing intrinsic dimensionality, although stitching charts of different dimensions requires explicit topology rules.

1.9.0.1 Implementation Status: not implemented — open experiment. `manifold_dim` is a single, fixed, caller-supplied `integer(int32)` argument threaded unchanged through the entire pipeline (`lomanle_pass`, `grow_adaptive_neighborhoods`, `grow_one_point_neighborhood`, `compute_anchor_svd`, `stitch_points`, ...). No subroutine inspects the eigenvalue spectrum to propose d_i locally, and there is no topology rule for stitching charts of different dimensions, since no chart ever has a dimension different from any other. This entire section remains exactly what it says it is — optional and unbuilt.

Lab notes. `misc/smoothing_experiments.md`'s "Open Questions" §5 ("Choosing k") is the closest existing lab-book discussion, and it is about $k_{\min}$, not d — it proposes (also unimplemented) a score `residual + overlap penalty + instability penalty` evaluated across a $k_{\min}$ sweep, choosing near the elbow. No equivalent sweep-and-score idea has been written down yet for d itself.

1.10 Phase IV — Local density / reliability labels

Each point receives a local density or reliability measure derived from its adaptive neighborhood.

For a Gaussian kernel,

$$w_{ij} = \exp\left(-\frac{\|\mathbf{x}_j - \mathbf{x}_i\|^2}{2\sigma_i^2}\right).$$

A simple local density estimate is

$$\rho_i = \sum_{j \in N_i} w_{ij}.$$

Alternatively a weighted-distance measure can be retained if that is the implementation used.

The adaptive bandwidth σ_i is derived from the local k-nearest-neighbor scale.

Because raw k-nearest-neighbor bandwidths can vary abruptly from point to point, the bandwidth field may itself be smoothed over neighboring observations before subsequent use.

Density/reliability is used primarily for **atlas construction and weighting**, not as a definition of the manifold itself.

1.10.0.1 Implementation Functions: computed inline inside `grow_one_point_neighborhood` (`src/lomanle.F90`) — there is no separate density subroutine; `density_i` is written on every growth step alongside the tangent/stability computation.

Formula actually used (matches `misc/smoothing_experiments.md` §5 exactly, and extends the formula above with an extra normalization not stated in this document):

$$\rho_i = \sum_j \exp(-d_j^2/(2\sigma_i^2)), \qquad \texttt{density\_i} = \rho_i / \max(\sigma_i, \epsilon)^{\texttt{manifold\_dim}}.$$

Complexity: folded into Phases II/III's cost above — no extra pass.

Status vs. this document — bandwidth smoothing is not implemented. "The bandwidth field may itself be smoothed over neighboring observations before subsequent use" (this doc) and the equivalent statement in `misc/smoothing_experiments.md` §18's refinement discussion are both aspirational; no smoothing of the σ_i field exists anywhere in `src/lomanle.F90`. **Open experiment.**

1.11 Phase V — Greedy atlas construction

A chart is defined by an anchor a, its adaptive neighborhood N_a, center $\boldsymbol{\mu}_a$, tangent basis U_a, and optionally local intrinsic dimension d_a.

Sort candidate anchors by decreasing reliability/density:

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(n)}.$$

Select the first high-density candidate as an anchor.

Subsequent anchors are chosen such that:

1. they add previously insufficiently covered observations;
2. neighboring charts retain a prescribed overlap;
3. already adequately represented regions do not generate redundant anchors.

The intended result is

$$\mathcal{A} = \{A_1, \ldots, A_m\}$$

covering the sampled manifold with overlapping local charts.

This anchor-selection procedure is presently an **algorithmic heuristic**, rather than part of the local tangent-space estimator itself.

1.11.0.1 Implementation

Functions: `construct_atlas` (orchestrator) -> `sort_points_by_density` (Step 4), `select_atlas_anchors` (Step 5), `absorb_orphans` (Step 5b), all `src/lomanle.F90`. `select_atlas_anchors` uses `kd_range_query_list`/`kd_range_query_mask` (kd_tree mod) for candidate-overlap and coverage queries; its per-candidate overlap-ratio evaluation is itself an `!$omp parallel do`.

Complexity: time $O(n_a\, n \log n)$ — this is, per the lab book, **the single largest remaining unresolved cost** in the whole pipeline (see below). Memory: $O(n)$ (`candidate_ratio`, `candidate_eligible`, `is_covered_mask`) plus $O(n)$ of per-thread private `range_buf_i(n_points)` scratch inside the parallel candidate-evaluation loop, i.e. $O(nT)$.

Lab notes — performance history (`misc/smoothing_experiments.md` §18.1–18.2, §18.7). Before 2026-07-27, Step 5's candidate-overlap check was $O(n_a\, n^2)$ (a brute-force distance loop per candidate per anchor pick). Replacing the brute-force scan with `kd_range_query_list`/`kd_range_query_mask` brought this to today's $O(n_a\, n \log n)$. That win **shrinks as the query radius grows and can vanish entirely**: a k-d tree only prunes a subtree when it can prove no point inside it is within the query radius, so once the radius already covers a large slice of n, the query degrades toward a full $O(n)$ scan — measured directly, at $k_{\min}$ values where anchor spheres already cover ~16% of the dataset, Step 5's speedup from this fix was within measurement noise of zero.

Open experiment — the dominant remaining cost. Step 5's greedy loop still re-scans *all* n remaining candidates on every one of the n_a anchor picks, because a candidate's overlap ratio can change whenever *any* nearby anchor gets picked, not just the most recently added one — so there is currently no cheap way to know which candidates are still "clean" without re-checking all of them. `misc/smoothing_experiments.md` §18.7 explicitly names an event-driven / priority-queue restructuring (only re-evaluate a candidate when a nearby anchor was actually just added) as the fix that could remove the n_a factor entirely, and flags it as needing careful design before attempting — "meaningfully more correctness risk than anything else in this section."

This was discussed at length in this session (2026-08-04) before the Tier-2 MST fix below was implemented: the concrete design is to (1) precompute each candidate's fixed sphere membership *once* via one k-d-tree pass (O(n log n) total, since `sphere_radii` is fixed before Step 5 starts), stored as a sparse CSR list, not a dense $n \times n$ mask (which would be infeasible — 2.5 billion entries at $n = 50{,}000$); (2) also build the transpose ("point -> candidates whose sphere contains it"); (3) when a new anchor is picked, use the transpose to incrementally update only the `num_overlap_i` counts of candidates actually affected by the newly covered points, instead of re-querying every candidate's sphere from scratch. This removes the n_a multiplier, turning $O(n_a\, n \log n)$ into roughly $O(n \log n)$ one-time build $+O(\text{total sphere occupancy})$ for incremental updates. **This has not yet been implemented** — it remains the single most valuable next step for Step 5's complexity, per the lab book's own ranking.

1.12 Phase VI — Recompute definitive chart geometry

For each selected anchor A_a, gather its final adaptive neighborhood

$$N_a.$$

Compute its final center

$$\boldsymbol{\mu}_a = \frac{1}{|N_a|} \sum_{i \in N_a} \mathbf{x}_i$$

and tangent basis

$$U_a \in \mathbb{R}^{D \times d_a}.$$

The local manifold chart is

$$M_a = \left\{ \boldsymbol{\mu}_a + U_a \mathbf{z} : \mathbf{z} \in \mathbb{R}^{d_a} \right\},$$

restricted to the local region represented by the chart.

1.12.0.1 Implementation Functions: `compute_anchor_svd` (`src/lomanle.F90`), an `!$omp parallel do` over anchors. Uses `kd_range_query_list` and `dsyev`.

Complexity: time $O(n_a \log n)$ (dominated by the per-anchor k-d-tree range query; the covariance accumulation is $O(\bar{k}_a \cdot \dim^2)$ and `dsyev` is $O(\dim^3)$ per anchor, both small relative to the query for realistic dim). Memory: $O(n)$ output arrays plus $O(n)$ of per-thread private `tmp_n_loc(n_points)` scratch, i.e. $O(nT)$.

Why this step exists at all (rather than reusing Steps 1-3's own neighborhood for the anchor point): some radii may have changed during atlas construction (Step 5) or the orphan pass (Step 5b), so an anchor's *final* sphere can differ from the neighborhood it was originally grown with. This step also recomputes `normal_errors` over the anchor's full final sphere — a broader, more reliable fit-quality measure than the narrower Steps 1-3 value — which Step 10's inverse-variance stitching weight (Phase XI below) depends on directly.

Lab notes — a real bug found here (`misc/smoothing_experiments.md` §18.2). The first version of this optimization (swapping a brute-force distance loop for `kd_range_query_list`) produced *different*, though still structurally valid, output from the original — not because the query itself was wrong (verified correct against brute force, zero mismatches), but because `kd_range_query_list` returns points in k-d-tree traversal order, not ascending index order, and the center/covariance summation is a floating-point accumulation — floating-point addition is not associative, so a different summation order gives a last-bit-different result, which then cascades through `lomanle_compute`'s outer convergence loop (often not fully converged within `max_iterations`) into a visibly different final skeleton. **Fixed by sorting the compact query result back to ascending point-index order before accumulating** — this is exactly why the current code contains the explicit insertion-sort block right after the `kd_range_query_list` call, with a comment recording this history. `select_atlas_anchors` and `build_membership_matrix` did not need the same fix, since they only ever produce integer counts or boolean masks (order-independent), not floating-point accumulations.

1.13 Phase VII — Project observations onto local manifolds

For observation $\mathbf{x}_i \in N_a$, define

$$\mathbf{v}_{ia} = \mathbf{x}_i - \boldsymbol{\mu}_a.$$

Its tangent coordinates are

$$\mathbf{z}_{ia} = U_a^\top \mathbf{v}_{ia}.$$

Its projection onto chart a is

$$\boxed{\hat{\mathbf{x}}_{ia} = \boldsymbol{\mu}_a + U_a U_a^\top (\mathbf{x}_i - \boldsymbol{\mu}_a)}$$

and its residual is

$$\mathbf{r}_{ia} = \mathbf{x}_i - \hat{\mathbf{x}}_{ia}.$$

By construction,

$$U_a^\top \mathbf{r}_{ia} = 0.$$

The scalar orthogonal reconstruction error is

$$r_{ia} = \|\mathbf{r}_{ia}\|_2.$$

Importantly, **the same original observation identifier i is retained for every chart projection**. Thus projections of the same observation into overlapping charts provide natural correspondences for stitching.

1.13.0.1 Implementation Status: fused into Phase XI, not a separate materialized step. There is no subroutine that computes and stores $\hat{\mathbf{x}}_{ia}$ for every (i, a) pair before stitching. Instead `stitch_multi_anchor_point`/`stitch_single_anchor_point` (Step 10, `src/lomanle.F90`) compute the projection for a given point against each of its covering anchors **inline**, immediately weight and accumulate it, and discard it — the per-chart projection $\hat{\mathbf{x}}_{ia}$ never exists as a standalone array. The math is identical to the boxed formula above (center + sum of tangent-direction projections); only the data-flow differs. This is a memory-saving fusion (no $O(n \cdot \text{anchor\_count})$ projection array is ever allocated), not a behavioral difference.

The retained-identifier property described in the last paragraph is exactly what makes this fusion possible in the first place: because `covering_anchors` (from Step 8's CSR point->anchor list) already carries point i's identity through to Step 10, no correspondence search is ever needed — see Phase XI below.

1.14 Phase VIII — Build the chart-overlap graph

Construct the membership relation

$$P_{ia} = \begin{cases} 1, & \mathbf{x}_i \in N_a, \\ 0, & \text{otherwise.} \end{cases}$$

Two charts are potential neighbors if

$$N_a \cap N_b \neq \varnothing.$$

Define the preliminary chart graph

$$G = (V, E),$$

where

$$V = \{A_1, \ldots, A_m\}$$

and

$$(A_a, A_b) \in E \iff N_a \cap N_b \neq \varnothing.$$

Sparse relations such as

- point → overlap edges,
- edge → shared points

are stored using **Compressed Sparse Row (CSR)** structures.

Connected overlap regions can then be identified using **Breadth-First Search (BFS)**.

However:

neighborhood overlap does not imply manifold connectivity

because overlapping spheres can represent:

- consecutive pieces of one manifold;
- a true bifurcation or multifurcation;
- intersecting manifolds;
- separate manifold branches passing close to each other.

Therefore sphere overlap creates only a **candidate stitching relation**.

1.14.0.1 Implementation Functions: `build_intersection_graph_alloc` (orchestrator, "Steps 6.5–9") -> `build_anchor_mapping`, `build_membership_matrix` (the dense P_{ia} matrix), `count_anchor_intersection_edges`, `build_point_anchor_lists`, `fill_anchor_intersection_edges`, `count_intersection_csr_sizes`, `build_intersection_gra` (fills the CSR structures and runs BFS), all `src/lomanle.F90`.

Complexity: time $O(n_a\, n)$. Memory: `point_in_anchor_mask` (the literal implementation of P_{ia} above) is a **dense** `logical(n_points, n_anchor)` array — $O(n\, n_a)$ memory. For $n = 50{,}000$ and a few hundred anchors this is tens of millions of entries (order 100MB at 4-byte logicals); not currently a measured problem, but the one dense structure in this pipeline whose memory actually scales with both n *and* n_a together, unlike Step 5 (Phase V) which was deliberately kept sparse, and unlike the MST's `pair_seen_mask(n_anchor, n_anchor)` (Phase XII), which is dense but only $O(n_a^2)$ — cheap since $n_a \ll n$.

Lab notes — performance history (`misc/smoothing_experiments.md` §18.3). Before 2026-07-27, finding anchor-pair overlaps was a literal implementation of "$\exists$ point in both N_a and N_b" checked over every $O(n_a^2)$ pair against all n points — $O(n_a^2\, n)$. The current implementation instead visits each **point** once, reads off the handful of anchors covering it directly from the membership matrix, and enumerates pairs only among those — using a `pair_seen_mask/edge_id_of_pair` lookup (the same technique the MST's Tier-1/Tier-2 candidate code uses, Phase XII below) so no $O(n_a^2)$ matrix ever needs an $O(n)$ inner scan. This dropped the cost to today's $O(n_a\, n)$. The same point-then-pair technique also replaced Step 10's per-point anchor lookup (Phase XI below).

Open experiment — the BFS output is not what actually decides connectivity. This is the most important divergence between this document and the code in the entire pipeline, and it is not obvious from reading the math above: `labels` (the BFS-computed connected-overlap-region id) **is computed, output, and used for diagnostic/visualization plots — but it does not drive any stitching or topology decision downstream.** `misc/smoothing_experiments.md`'s own remaining-work summary (§18, item 2) states this explicitly: "the anchor-graph/MST approach does not use the BFS intersection labels for connectivity at all (only the plain point-to-anchor pairing and, as a fallback, sphere overlap); the BFS labels remain useful as a diagnostic/visualization grouping ... just not as the connectivity rule." Actual connectivity is decided in Phase XII via the anchor MST (Tier-1 topological pairing from Step 10's primary/secondary anchors, Tier-2 geometric sphere-overlap fallback) — a genuinely different mechanism from "grow BFS-connected overlap regions" as this document's boxed statement and §14 originally implied. The boxed principle itself ("neighborhood overlap does not imply manifold connectivity") remains exactly correct — it is *why* labels alone were never sufficient — but the *resolution* mechanism this document originally sketched (Phase IX/X below) was not what got built; a different, MST-based resolution was.

1.15 Phase IX — Estimate the local normal/noise structure

For each chart a, use residual vectors

$$\mathbf{r}_{ia}$$

to estimate the distribution of observations normal to the local manifold.

Let

$$U_a^{\perp}$$

span the orthogonal complement of U_a.

Normal coordinates are

$$\mathbf{q}_{ia} = (U_a^{\perp})^{\top} \mathbf{r}_{ia}.$$

Estimate their covariance

$$\Sigma_{\perp,a} = \mathrm{Cov}\{\mathbf{q}_{ia} : i \in N_a\}.$$

This represents the local anisotropic noise/thickness around the manifold.

A scalar tube is the simpler special case

$$R_a = Q_q\left(\{\|\mathbf{r}_{ia}\|\}\right),$$

where Q_q is a robust local quantile such as the 95th percentile.

The covariance representation is preferable when sufficient data are available because normal variation need not be isotropic.

1.15.0.1 Implementation Status: partially implemented — the scalar special case only, not the anisotropic covariance. Open experiment.

`compute_anchor_svd` (Phase VI) computes exactly one scalar per anchor, `normal_errors(a)` — the *mean squared* residual over the anchor's full final sphere:

$$\texttt{normal\_error}_a = \frac{1}{|N_a|} \sum_{i \in N_a} \left(\|\mathbf{r}_{ia}\|^2\right).$$

This is close to, but not identical to, the "scalar tube" special case R_a above: the code uses a **mean squared residual**, not a robust quantile (Q_q, e.g. the 95th percentile) of the residual norms. No anisotropic $\Sigma_{\perp,a}$ (covariance of normal-space residuals $\mathbf{q}_{ia}$) is ever computed — only the isotropic scalar. $\Sigma_{\perp}$ and $U_a^{\perp}$ do not appear anywhere in `src/lomanle.F90`.

This is a live design question, not a stale one. This exact gap — "we only have a scalar reconstruction error per anchor; we don't retain the anisotropic residual structure" — was the subject of an extended design discussion in this session (2026-08-04), proposing exactly the $\Sigma_{\perp,a}$ structure this document already specified: a per-anchor array indexed like `tangent_bases/normal_errors` are today, holding either the raw normal-space residual vectors or their (dim −`manifold_dim`) × (dim −`manifold_dim`) covariance, to be used for (1) a genuine multi-furcation decision (Phase X below) and (2) potentially informing subsequent sphere/radius estimation. The tradeoff identified: full $\Sigma_{\perp,a}$ costs $O(n_a \cdot (\dim - \texttt{manifold\_dim})^2)$ memory versus the current $O(n_a)$ scalar, and requires changing `compute_anchor_svd`'s residual accumulation from a running scalar sum to an outer-product accumulation (still a legal `pure` SK computation). **Not yet implemented.**

1.16 Phase X — Decide whether candidate charts should actually be stitched

For every candidate chart pair (a, b) from the overlap graph, determine their closest relevant manifold locations

$$\mathbf{m}_a \in M_a, \qquad \mathbf{m}_b \in M_b.$$

Define

$$\boldsymbol{\delta}_{ab} = \mathbf{m}_b - \mathbf{m}_a.$$

The charts are eligible for stitching only if their local manifold models approach each other within the noise/thickness supported by the original observations.

In the scalar approximation,

$$\|\boldsymbol{\delta}_{ab}\| \leq R_a + R_b$$

provides a simple tube-overlap condition.

With anisotropic residual models, evaluate the separation relative to the combined normal uncertainty. Conceptually,

$$\Sigma_{\perp,ab} = \Sigma_{\perp,a} + \Sigma_{\perp,b},$$

after expressing both covariance structures in a compatible ambient-space representation.

Then evaluate a generalized Mahalanobis-type separation

$$D^2_{ab} = \boldsymbol{\delta}_{ab}^{\top} \Sigma^{+}_{\perp,ab} \boldsymbol{\delta}_{ab},$$

where + denotes the Moore-Penrose pseudoinverse because normal covariance is generally rank deficient in ambient coordinates.

Require

$$D^2_{ab} \leq \tau_{\text{noise}}.$$

Thus:

> Two charts that merely pass nearby are kept separate if the distance between their manifolds exceeds the locally observed residual/noise structure.

1.16.0.1 Implementation Status: not implemented — open experiment. There is no noise-tube or Mahalanobis test anywhere in `src/lomanle.F90`. The actual gating condition for whether a point's overlapping charts get stitched is purely topological: `stitch_points` (Step 10) calls `stitch_multi_anchor_point` whenever `anchor_count(i) >= 2`, unconditionally — i.e. *any* point covered by two or more anchor spheres gets blended between them, regardless of how far apart those charts' local manifold models actually are. The distance/noise gating this section describes plays no role in the current decision.

Lab notes — this exact idea is recorded verbatim in the lab book, as an explicitly unimplemented proposal (`misc/smoothing_experiments.md` §15, "Potential Stitching Experiment"):

> Identify for all manifolds (spheres) involved in local stitching the noise structure around the manifolds. Noise structure here is information contained in the residuals, i.e. the orthogonals between the manifold and the ambient space vectors. If the manifolds pass each other within their respective noise regions, assume a bi- or multi-furcation and stitch. If not, assume they are separate manifolds that just pass each other close by.

That section also records an AI-assisted design sketch (attributed there as "ChatGPT said on this") arguing for precisely the $\Sigma_{\perp}$/Mahalanobis approach this document already specifies mathematically — each chart's normal covariance defining an anisotropic tube, tested via combined-covariance Mahalanobis distance, with the explicit caveat that "noise-tube overlap should be necessary, but probably not sufficient" — geometric compatibility (tangent-angle structure, §15 below) should also be required. **This has not been implemented**, and depends on Phase IX's $\Sigma_{\perp,a}$ first existing.

What the code does instead, as a substitute. Rather than gating *whether* to stitch, the code gates *how strongly* each anchor's projection contributes once stitching has already been decided (unconditionally) to happen — via the inverse-variance weight $1/\texttt{normal\_error}_a$ (Phase XI below). A poorly-fit anchor is down-weighted, not excluded. This is a real difference in kind, not just in strictness: the current mechanism can never produce "these two charts are unrelated, do not combine them at all" — it can only ever produce "combine them, but trust one more than the other."

1.17 Tangent compatibility

Noise-tube overlap alone is insufficient.

For charts a, b, compute principal angles between their tangent spaces.

For equal dimension d,

$$\cos\theta_j = s_j(U_a^\top U_b),$$

where s_j are the singular values.

These angles distinguish possible relationships.

1.17.1 Continuation

Small principal angles indicate that two charts likely represent consecutive pieces of the same smooth manifold.

1.17.2 Furcation

Charts whose noise-supported manifold regions meet but whose tangent structures diverge may represent a bifurcation or multifurcation.

Such a furcation should be introduced only when:

1. the local noise regions support physical contact;
2. original observations populate the junction region — the **non-gap requirement**;
3. tangent geometry supports multiple outgoing manifold directions — the **angle requirement**.

1.17.3 Near miss

If anchor spheres overlap but manifold noise regions do not, do **not** stitch.

Therefore the conceptual rule is

$$\boxed{\text{stitch} = \text{local support} \wedge \text{noise-region compatibility} \wedge \text{topological/tangent compatibility}.}$$

The exact thresholds for distinguishing continuation, furcation and crossing remain parameters requiring empirical validation.

1.17.3.1 Implementation Status: not implemented as a per-chart-pair geometric test — open experiment. No subroutine computes principal angles $\cos\theta_j = s_j(U_a^\top U_b)$ between two *different* anchors' tangent bases anywhere in `src/lomanle.F90`. The nearest relative is inside `grow_one_point_neighborhood` (Phase II/III), which does compute a principal-angle-like stability measure — but only between a *single point's own* tangent basis across two consecutive growth steps (used to decide when to stop growing that one point's neighborhood), never between two distinct anchors' charts.

Furcation is instead detected structurally, after the fact, from the backbone MST (Phase XII). An anchor's degree in the minimum spanning tree built over the anchor graph classifies it: degree ≤ 1 = endpoint, degree $= 2$ = pass-through, degree ≥ 3 = branch/junction (`classify_anchor_roles`, `src/lomanle.F90`). This *achieves* a version of the furcation/continuation/near-miss classification this section describes, but by a completely different route — graph topology of accepted candidate edges (Tier-1: Step 10's primary/secondary anchor pairing; Tier-2: raw sphere overlap), not principal angles between tangent spaces, and not any noise-region test. The **non-gap requirement** ("original observations populate the junction region") and the **angle requirement** ("tangent geometry supports multiple outgoing directions") from this section are not separately checked; a junction is accepted whenever the MST happens to give an anchor degree ≥ 3, which can be driven purely by geometric proximity (Tier-2) with no direct evidence that the tangent geometry itself supports branching.

Lab notes — validated where it has been tested, with named gaps. `misc/smoothing_experiments.md`'s "Open Questions" §3 records that `bifurcation_2way` and `bifurcation_3way` synthetic datasets have been run end-to-end, in both 2D and 3D, at low/medium/high noise, and that `anchor_role` correctly marks the branch

point as a junction, with the rendered backbone showing a genuine Y-shaped split matching the dataset's known structure — and that the entire three-round backbone debugging process (Phase XII below) was driven by exactly these bifurcation datasets exposing real bugs, not by hypothetical concerns. **Explicitly still open:** T-shaped structures, more than one branch point in the same dataset, and deliberately uneven-density bifurcations have not been specifically tested; the fixes described in Phase XII "should generalize… but that is an assumption, not something confirmed on those specific shapes."

1.18 Phase XI — Stitch projections from overlapping charts

Suppose observation i belongs to charts

$$A_{a_1}, \ldots, A_{a_s}.$$

It then has chart-specific projections

$$\hat{\mathbf{x}}_{ia_1}, \ldots, \hat{\mathbf{x}}_{ia_s}.$$

Because all projections retain original point identifier i, no correspondence search is necessary.

The simplest stitched position is

$$\hat{\mathbf{x}}_i = \frac{1}{s} \sum_{j=1}^{s} \hat{\mathbf{x}}_{ia_j}.$$

Preferably use chart reliability weights,

$$\hat{\mathbf{x}}_i = \frac{\sum_j w_{ia_j} \hat{\mathbf{x}}_{ia_j}}{\sum_j w_{ia_j}}.$$

Weights may incorporate:

- local density;
- projection residual;
- tangent stability;
- local noise variance;
- distance from the chart center.

An inverse-variance form is

$$w_{ia} \propto \frac{1}{\sigma_{ia}^2 + \epsilon}.$$

Only chart projections belonging to a geometrically accepted common manifold component are combined.

1.18.0.1 Implementation Functions: `stitch_points` (orchestrator, "Step 10", `!$omp parallel do` over points) -> `stitch_multi_anchor_point` (`anchor_count(i) >= 2`), `stitch_single_anchor_point` (`anchor_count(i) == 1`), all `src/lomanle.F90`. Reads the CSR point->anchor list built in Phase VIII (`build_point_anchor_lists`).

Complexity: time $O(n \cdot \overline{\texttt{anchor\_count}}) \approx O(n)$, since `anchor_count(i)` is typically 2-3, not n_a — this is exactly the Phase VIII performance fix (point-then-pair CSR lookup, not a scan of all n_a anchors per point) applied here too. Memory: $O(n)$.

Status vs. this document — the weight is per-anchor, not per point-anchor pair. The design's $w_{ia} \propto 1/(\sigma_{ia}^2 + \epsilon)$ notation allows the weight to vary by *both* the point i and the anchor a. The implemented weight is

$$w_a = \frac{1}{\texttt{normal\_error}_a + \epsilon},$$

a function of the anchor alone (Phase VI's anchor-level fit quality) — every point covered by anchor a uses the *same* weight for that anchor, regardless of the point's own position within the sphere or its own individual residual. This is a deliberate simplification, not an oversight — see the lab notes below for why simpler alternatives were tried and rejected in favor of this one.

Lab notes — weighting scheme evolution (`misc/smoothing_experiments.md` §15). Three schemes were tried, in order:

1. **Density-weighted** (stitched $= \frac{\sum_k \text{density}_k \cdot \text{projection}_k}{\sum_k \text{density}_k}$): denser local manifolds contribute more. In practice this sometimes produced several visible lines instead of collapsing the overlap into one clean local trend, because anchors with moderately similar densities still contributed similar weights, so competing local projections could survive the average.
2. **Density^4-weighted**: made the densest sphere dominate more sharply. This helped but was "still not quite right" — density alone says nothing about tangent-plane fit quality; a small density advantage between two nearby anchors doesn't mean one of them actually fits the local geometry better.
3. **Current: inverse-variance by anchor fit quality** ($w_a = 1/\texttt{normal\_error}_a$). The idea: trust the anchor whose tangent plane genuinely fits its neighborhood, regardless of how many nearby points it has. This collapses ordinary overlap points onto a single central trend while still letting genuine bifurcations (where two anchors fit about equally well) land roughly between both branches instead of being forced onto one.

As a side effect, `stitch_multi_anchor_point` also records which two anchors give the **highest and second-highest** weight for each point (`primary_anchor_ids`, `secondary_anchor_ids`) — not just to blend the position, but because that same pair is exactly what Phase XII's backbone construction (Tier-1 candidate edges) uses to build the anchor-level topology graph.

"Only chart projections belonging to a geometrically accepted common manifold component are combined" (this document's last line) **is not currently enforced** — see Phase X above: stitching happens unconditionally for any `anchor_count(i) >= 2`, with no geometric-acceptance gate.

1.19 Phase XII — Construct explicit manifold topology

The stitched projected points

$$\hat{X}$$

are samples from the estimated manifold, but are not by themselves an explicit manifold representation.

For intrinsic dimension $d = 1$, construct a graph connecting neighboring manifold points while respecting the accepted chart topology.

This yields a piecewise-linear principal curve/skeleton.

For $d = 2$, the corresponding representation requires faces, e.g. local triangulation, constrained by chart membership and accepted chart adjacency.

For general d, the analogous representation is a local simplicial complex or atlas representation.

Crucially, nearest Euclidean projected points must **not** automatically be connected, because folds can make geodesically distant manifold regions close in ambient space.

Connectivity should therefore derive primarily from the accepted chart graph and local manifold coordinates.

The current $d = 1$ connectivity algorithm exists; generalization to $d > 1$ remains an explicit development task.

1.19.0.1 Implementation Status: implemented for $d = 1$ — but via a substantially different mechanism than "connect neighboring manifold points while respecting chart topology" might suggest.

The actual method builds and threads a graph over **anchors**, not over points directly, and the point-level chain only falls out at the very end.

Functions: `build_skeleton_edges_alloc` (orchestrator, "Step 11" in the lab book) calling, in order: - `build_anchor_mapping` — compact <-> original point index mapping. - `build_anchor_mst_alloc` -> `mark_tier1_candidate_pairs` (Tier 1: candidate anchor-pair edges from Phase XI's `primary_anchor_ids/secondary_anchor_i` `count_tier2_candidate_pairs` (Tier 2 sizing), `build_anchor_mst` -> `build_tier1_mst_edges`, `build_tier2_mst_edges` (Kruskal + union-find over both tiers), `classify_anchor_roles` (degree -> endpoint/pass-through/junction). - `build_member_chains` — threads every point into its *primary* anchor's local chain, ordered by projection onto that anchor's tangent axis. - `build_branch_adjacency` — CSR adjacency of the anchor MST. - `build_skeleton_edges` -> `emit_branch` (walks each maximal run of pass-through anchors between two "special" anchors as one branch, giving every point on it one shared, monotonically increasing coordinate), `nearest_member`, `find_root` (union-find with path compression).

Complexity: time $O(n_a \log n_a)$ for the MST (as of the fix below) + $O(n \log(n/n_a))$ for the per-anchor/per-branch member-chain sorts. Memory: `pair_seen_mask(n_anchor, n_anchor)` is dense but only $O(n_a^2)$ — cheap, since $n_a \ll n$ — plus $O(n)$ for the member/branch/edge buffers.

Why not "connect nearest points" (`misc/smoothing_experiments.md` §16.1). The first implementation of this step did exactly that: each point greedily picked its own nearest tangent-aligned successor, with fallback stages and an explicit whole-graph walk before accepting each candidate edge to avoid closing a loop. This fights the shape of the actual problem in two structural ways: (1) Euclidean closeness is not the same as belonging to the same branch — two points on nearby-but-different branches can be closer to each other than to their own branch's continuation, and a rule based on distance and local tangent alignment alone cannot tell the difference; (2) a "next point" model gives every point at most one outgoing edge, but a genuine branch point structurally needs three or more, requiring bolted-on bookkeeping (a "convergence point" special case inferred by counting incoming edges) rather than falling naturally out of the data structure. **The core idea that replaced it: the topology lives on the anchors, not on individual points** (`misc/smoothing_experiments.md` §16.2) — Phase XI already records, for every overlap point, exactly which two anchors it blends between; whenever a point uses two anchors as primary+secondary, those anchors are, by construction, part of the same connected piece of the manifold. That is a topological fact already sitting in memory, not something requiring re-derivation point by point.

Lab notes — three attempts, three bugs found and fixed (`misc/smoothing_experiments.md` §16.7), turning the anchor-level tree into an actual point-level polyline:

1. **Per-anchor local chain + single bridge point.** Every point was threaded into the chain of *both* its primary and secondary anchor, sorted within each by its own local tangent axis. *Bug:* a point in the overlap of two anchors got ordered independently in two chains using different, disagreeing axes — visible as short, duplicated, overlapping fragments at every chart boundary, confirmed by `n_edges` coming out far above `n_nodes - 1` for what should have been a tree. *Fix:* thread every point into exactly **one** chain — its primary anchor's only.
2. **One chain per anchor, still ordered locally.** With duplication fixed, a visible zig-zag/staircase texture appeared, especially where several short-radius anchors sit close together: each anchor's tangent line is estimated from a small, noisy neighborhood, so two adjacent anchors' lines can disagree slightly in orientation, and drawing "all of anchor A, then all of anchor B" prevented genuinely interleaved points near the shared boundary from interleaving. *Fix:* stop resetting to a local coordinate at every chart. Walk each maximal run of pass-through anchors between two special anchors as one **branch**, giving every point on it one shared, monotonically increasing coordinate (each anchor's tangent axis oriented "forward" along the branch, plus a running center-to-center offset). Sorting the *whole branch* by this shared coordinate lets points near a boundary interleave correctly even when neighboring anchors' local axes disagree. A point that also blends into a secondary anchor on the same branch gets the *average* of both anchors' coordinate values — the same smoothing role Phase XI's position blending already plays, applied to ordering too.
3. **Branches naturally re-threading their own endpoints.** The first branch-based implementation pooled *every* anchor along a branch's path, including both special (junction/endpoint) ends, into that branch's own sorted chain — so a junction's member points got pooled and re-sorted once *per incident branch*. Same edge-count diagnostic (`n_edges` vs. `n_nodes - 1`) showed dozens of redundant edges concentrated exactly at junctions and endpoints. *Fix:* thread every special anchor's own member cluster into its own simple local chain exactly **once**; have each branch pool only its *interior* (pass-through) anchors' points, bridging each end

into its special anchor's nearest member.

Result (`misc/smoothing_experiments.md` §16.8): on `bifurcation_2way`, the final-stage graph is a single connected component with `n_edges == n_nodes - 1` (a true tree, no residual cycles) — a handful of near-zero-length edges can still appear where two independently-stitched points collapse onto almost the same position at a hub, harmless and invisible at plotting scale. Rendered, it shows one continuous line with a genuine Y-shaped bifurcation matching the dataset's known structure.

Open, not started — the $d > 1$ mesh case. This document's own statement ("the current $d = 1$ connectivity algorithm exists; generalization to $d > 1$ remains an explicit development task") is still exactly accurate. `misc/smoothing_experiments.md` §16.12 spells out the gap precisely: Step 10's stitched point positions already respect `manifold_dim` fully (a `manifold_dim=2` run on 3D input genuinely projects each point onto its local 2-D tangent plane), so a denoised point cloud lying on the surface is already available in `skeleton_coords`. But `build_skeleton_edges` is 1-D **by construction**, regardless of `manifold_dim`: the candidate-edge tiers, the MST, and the branch-threading all produce a single ordering coordinate per anchor via `tangent_bases(:, 1, k)` — only the *first* tangent direction — because the whole design is "thread points into a chain along a branch." A chain has no notion of a second ordering axis, no face/triangle structure, and no code path letting three or more points at the same "branch position" form a 2-D patch instead of colliding onto one polyline vertex. Getting an actual mesh (faces connecting anchors in two directions, not a spanning tree) needs new design work — e.g. connecting anchors along a second independent tangent direction, or triangulating within/across overlapping charts — not a reuse of the MST/branch machinery as-is.

Performance lab notes — the Tier-2 MST fix (2026-08-05, this session). `count_tier2_candidate_pairs`/`build_tier2_m` used to enumerate every anchor pair not already flagged by Tier 1 via an $O(n_a^2)$ double loop, checking `dist(a,b) \le r_a + r_b` for each. `misc/smoothing_experiments.md` §18.7 had flagged this as "cheap in every case measured so far (n_anchor stayed in the hundreds), but a real $O(n_a^2)$ term if n_a ever gets much larger" and left it undone. It is now fixed: `build_anchor_mst_alloc` builds a small second k-d tree over just the n_a anchor centers (reusing `build_kd_index`), and both routines query it per-anchor with radius $r_a + \max_b r_b$ — a provably conservative bound, since every other anchor's radius is $\le \max_b r_b$, so no true overlap is ever missed — then apply the same exact `dist \le r_a + r_b` filter as before. This is $O(n_a \log n_a)$ instead of $O(n_a^2)$. **Verified bit-identical** to the pre-fix output on five dataset/parameter combinations (`bifurcation_2way_noise_medium` k=30; `bifurcation_3way_noise_low` $k \in \{15, 40\}$; `circular_arc_noise_low` $k \in \{15, 40\}$ — the last chosen specifically because its symmetric geometry is where an enumeration-order-dependent tie-break would most plausibly show up), including the one documented, expected-benign caveat: since `sort_array` (quicksort) is not a stable sort, two anchor pairs with *exactly* equal center-distance could in principle break a Kruskal tie differently than the old enumeration order; this did not occur in any tested case and is a measure-zero event for real-valued distances.

1.20 Phase XIII — Iterative manifold refinement

The stitched manifold can retain small discontinuities, roughness and chart-boundary artifacts.

Apply iterative manifold learning/smoothing to the stitched projected points.

The important principle is that refinement should reduce noise without destroying the geometry recovered by the atlas.

For each manifold point $\hat{\mathbf{x}}_i$, obtain an adaptive local neighborhood and Gaussian weights

$$w_{ij} = \exp\left(-\frac{\|\hat{\mathbf{x}}_j - \hat{\mathbf{x}}_i\|^2}{2\sigma_i^2}\right).$$

The bandwidth field σ_i may itself be smoothed to avoid abrupt changes between neighboring kernels.

The update produces a new manifold estimate

$$\hat{X}^{(t+1)} = F(\hat{X}^{(t)}).$$

The previously developed iterative ManLe/ANWIL-style procedure can provide F.

A critical implementation choice is whether the current point contributes to its own update. Excluding it entirely can produce excessive displacement in sparsely supported regions; therefore this choice should be treated explicitly and validated.

Likewise, hard truncation at exactly the k-th neighbor can introduce discontinuities as observations enter or leave adjacent kernels. Full Gaussian support, efficient radius truncation at negligible weights, or smoothly varying neighborhoods should be considered.

1.20.0.1 Implementation Status — F is not an external ManLe/ANWIL-style smoothing call; it is the whole atlas-and-stitch pipeline applied to its own output.

`lomanle_compute` (`src/lomanle.F90`) runs:

```
initial coordinates -> lomanle_pass -> stitched coordinates -> lomanle_pass -> stitched coordinates -> ...
```

i.e. $F(\hat{X}^{(t)})$ is literally "rebuild the local atlas from scratch over $\hat{X}^{(t)}$ and restitch" (Phases I–XII again, in full), not a call into `src/manle_module.F90` (ManLe/AManLe) or `src/anwil.F90` (ANWIL) as this section's wording suggests was the plan. `tangent_bases` is warm-started from the previous outer iteration (zero on the first call) to seed Phase III's stability check for the smallest neighborhood on the next pass.

Functions: `lomanle_compute` (outer loop) repeatedly calling `lomanle_pass` (`src/lomanle.F90`); no functions from `manle_module.F90`/`anwil.F90` are called anywhere in this path.

Complexity: the cost of one full `lomanle_pass` (i.e. everything in the table under "Implementation status at a glance" above) $\times$ the number of outer iterations actually run ($\leq$ `max_iterations`). Memory: an additional $O(n)$ for the iteration-1 snapshot copies (`skeleton_iter1`, `radii_1`, `densities_1`, `gap_1`, `is_anchor_mask_1`, `labels_1`, `tangent_bases_1`, `tangent_scales_1`, `primary_anchor_1`, `secondary_anchor_1`, `anchor_centers_1`), captured once, not per iteration.

The "does the current point contribute to its own update" and "hard truncation at exactly k" questions this section raises are moot in this implementation — they are ManLe/ANWIL-specific concerns (those methods do define F as an explicit local-kernel-weighted update over a *fixed* point cloud) that do not apply to LoManLe's actual F, which rebuilds the entire adaptive atlas (including re-growing every neighborhood from scratch) rather than applying one fixed smoothing kernel.

Lab notes — this iterative structure is the key design choice, and it is validated, not merely convenient. This was stated directly in this session (2026-08-04): the iterative re-collapse of stitched points toward the learned manifold is important — without it, earlier (non-iterative) versions of the pipeline produced multiple parallel or nearby spurious manifolds instead of one clean skeleton. `misc/smoothing_experiments.md` §2 documents the mechanics (`max_disp` displacement check, `conv_tol` derived as `relative_conv_tol × median nearest-neighbor distance` in the *original* coordinates so the same `relative_conv_tol` behaves sensibly regardless of the data's own scale) and §18.8-18.9 give the measured wall-clock cost of running this loop for up to 50 iterations across problem sizes from $n = 500$ to $n = 50{,}000$, at both 12 and 128 threads.

1.21 Stop refinement before oversmoothing

At each iteration measure at least:

- roughness r_t;
- coverage c_t;
- reconstruction error, e.g. root mean squared error e_t.

A geometric mean of these quantities was experimentally found unsuitable because it continued refinement until the maximum iteration count.

A weighted arithmetic objective performs better:

$$L_t = w_r \, \tilde{r}_t + w_c \, \tilde{c}_t + w_e \, \tilde{e}_t,$$

where quantities must be normalized to comparable scales and their orientation chosen so that smaller L consistently means better.

Experiments indicate useful relative emphasis around

$$w_r : w_c \approx 2 : 3$$

or

$$3 : 2,$$

depending on the dataset.

Refinement stops when further iteration no longer improves the objective according to the defined stopping rule.

The exact objective and stop condition are empirical hyperparameters and should not yet be treated as universal constants.

1.21.0.1 Implementation Status: not implemented for LoManLe's own outer loop — open experiment.

`lomanle_compute`'s actual stopping rule is a single displacement check:

```
max_disp = max_i || skeleton_coords(i) - work_coords(i) ||
converged = max_disp < conv_tol
```

No roughness, coverage, or RMSE quantity is computed anywhere in `src/lomanle.F90`, and there is no weighted objective L_t. On non-convergence within `max_iterations`, the code emits a warning (currently via `print`, flagged as an F42 "no console I/O in SK routines" violation with a `TODO` comment as of this session) rather than falling back to any quality-based early stop.

Where this experimental result actually comes from. This section's specific findings — the geometric mean being unsuitable, the $w_r : w_c \approx$ 2:3 or 3:2 relative emphasis — are recorded in `misc/smoothing_experiments.md`'s introduction as belonging to the broader family of smoothing methods tested there (Loess, Nadaraya–Watson, ANWIL, ManLe, AManLe — see that file's `run_smoothing_tests.sh` usage, which accepts `score_list/w_rough/w_rmse/w_cov` arguments precisely for this kind of weighted objective, `method_id` 1 through 7). **This machinery has not been ported to `lomanle_compute`'s own convergence loop.** Whether LoManLe's iterative atlas-rebuild-and-restitch (a structurally different F than any of those methods', per Phase XIII above) needs the same kind of quality-weighted early stop, or whether the simpler `max_disp` check is sufficient because the atlas-rebuild dynamics behave differently, is itself an open question this document did not previously distinguish.

Open experiment: define and wire up roughness/coverage/reconstruction-error tracking for `lomanle_pass`'s own iteration sequence, decide whether the ManLe/AManLe weighting lessons ($w_r : w_c \approx$ 2:3) transfer, and replace or augment the current `max_disp`-only stop.

1.22 Complete algorithm

```
LOMANLE(X, parameters)

    # ------------------------------------------------------------
    # A. Local geometry                                [Phases I-IV; see §4-8]
    # ------------------------------------------------------------

    build nearest-neighbor index for X

    for every observation i:

        k <- k_min

        repeat:

            N <- kNN(x_i, k)

            estimate local center mu
            estimate local covariance / SVD
```

```
        estimate tangent subspace U_d

        if intrinsic dimension is adaptive:                    # NOT IMPLEMENTED (§7)
            infer candidate d from spectral structure

        compare tangent structure with previous scale
        [actual rule: quality score = gap + stability - normal_error - radius,
         keep best-so-far, not last-evaluated -- see §6 Implementation]

        if tangent structure is stable
           AND tangent/normal spectral separation is sufficient:
               accept neighborhood
               break

        k <- ceil(g * k)

    until k >= k_max

    store:
        neighborhood
        local scale
        density/reliability
        tangent stability
        intrinsic dimension
        spectral information


optionally smooth the local bandwidth/scale field         # NOT IMPLEMENTED (§8)


# ------------------------------------------------------------
# B. Atlas construction                                 [Phase V; see §9]
# ------------------------------------------------------------

rank candidate anchors by density/reliability

greedily select anchors such that:
    manifold coverage increases
    required overlap is maintained
    unnecessary redundant charts are avoided
[dominant remaining cost: O(n_anchor . n . log n),
 event-driven restructuring identified but not yet implemented -- see §9]

for each selected anchor:
    recompute definitive neighborhood
    compute center mu_a
    compute tangent basis U_a
    project member observations onto local manifold
    retain original observation identifiers
    compute residual vectors


# ------------------------------------------------------------
# C. Candidate topology                               [Phase VIII; see §12]
# ------------------------------------------------------------
```

```
construct point x anchor membership

create candidate edge (a,b)
    whenever charts share observations

store sparse incidence structures using CSR

identify connected overlap regions using BFS
[these labels are diagnostic only -- NOT used for the connectivity
 decision below; see §12 Implementation, "Open experiment"]


# ------------------------------------------------------------
# D. Geometric topology                    [Phases IX/X + Tangent compat.; see §13-15]
# ------------------------------------------------------------
# NOT IMPLEMENTED AS DESCRIBED -- see §13-15. What actually runs instead
# is anchor-graph MST construction (Tier 1: Step 10's own primary/
# secondary pairing; Tier 2: raw sphere-overlap fallback), described in
# block F' below. No noise-tube, Mahalanobis, or tangent-angle test
# exists in the code.

for every candidate chart edge (a,b):

    estimate residual/noise structure of chart a           # only a scalar
    estimate residual/noise structure of chart b           # exists (§13)

    determine closest relevant manifold regions

    if manifold regions do not overlap within
       their supported noise regions:                      # NOT CHECKED
           reject edge
           continue

    analyze tangent-space compatibility                     # NOT CHECKED

    if geometry supports continuation:
           classify as continuation edge

    else if:
           observations support the junction
           AND tangent geometry supports branching:
           classify as furcation edge

    else:
           reject or mark unresolved


# ------------------------------------------------------------
# E. Stitching                                  [Phase XI; see §16]
# ------------------------------------------------------------

for every original observation represented
in multiple compatible charts:                            # "compatible" = anchor_count>=2,
                                                            # unconditional -- see §14

    collect its chart-specific projections
```

```
    combine projections using
    reliability / inverse-variance weights
    [actual weight: 1 / anchor's own normal_error, same for every
     point the anchor covers -- not a per-point sigma_ia; see §16]

    obtain one stitched manifold point


# ------------------------------------------------------------
# F. Explicit topology                                [Phase XII; see §17]
# ------------------------------------------------------------

if d == 1:
    [actual mechanism: build a graph over ANCHORS (not points),
     Tier-1 candidates from Phase XI's primary/secondary anchor pairs,
     Tier-2 candidates from raw sphere overlap (small anchor-only
     k-d tree, O(n_anchor log n_anchor) as of 2026-08-05), Kruskal MST,
     anchor role = degree in MST, THEN thread points into branches --
     see §17 for the full three-attempts-three-bugs history]
    connect manifold points into edges using
    local coordinates + accepted chart topology

if d > 1:                                           # NOT IMPLEMENTED (§17)
    construct local faces/simplices from chart topology
    and local manifold coordinates


# ------------------------------------------------------------
# G. Refinement                                     [Phase XIII; see §18-19]
# ------------------------------------------------------------

repeat:

    perform adaptive manifold smoothing
    [actual F: rebuild the ENTIRE atlas (blocks A-F above) over the
     current stitched points and restitch -- not an external ManLe/
     ANWIL call; see §18]

    evaluate:
        roughness                                    # NOT COMPUTED (§19)
        coverage                                     # NOT COMPUTED (§19)
        reconstruction error                         # NOT COMPUTED (§19)

    compute weighted objective                       # NOT IMPLEMENTED (§19)

    if stop criterion reached:
        [actual criterion: max_i||stitched_i - previous_i|| < conv_tol,
         where conv_tol = relative_conv_tol * median NN distance in the
         ORIGINAL coordinates -- see §18]
        break

until max_iterations


return manifold points,
```

```
manifold topology,
atlas,
local tangent models,
residual/noise models,
diagnostics
```

1.23 What is fundamental LoManLe and what is currently heuristic

It is useful to make this distinction explicit.

1.23.1 Geometric core

The following has a direct mathematical interpretation:

$$\boxed{\text{local neighborhood} \rightarrow \text{local SVD} \rightarrow \text{tangent subspace} \rightarrow \text{orthogonal projection}}$$

For a sufficiently smooth underlying manifold and sufficiently local neighborhoods, this estimates its first-order local geometry.

Implementation: exact — Phases I-III and VI-VII (§4-6, §10-11), functions `build_kd_index`, `grow_adaptive_neighborhoods/g` `compute_anchor_svd`. No divergence.

1.23.2 Adaptive estimation

Growing neighborhoods until tangent structure becomes stable addresses the variance-versus-curvature trade-off:

$$\text{too small} \rightarrow \text{unstable/noisy tangent}$$

$$\text{too large} \rightarrow \text{curvature/branch mixing.}$$

The smallest stable neighborhood is therefore the desired operating point.

Implementation: the *principle* is exact; the *acceptance rule* was substantially redesigned from a literal reading of §6 into a best-scoring-candidate-with-self-calibrating-patience rule — see §6 Implementation for the full rationale and the concrete failure modes of the naive rule it replaced.

1.23.3 Atlas construction

Anchor selection is presently heuristic. Its purpose is computational and representational: obtain sufficient overlapping local models without fitting a chart around every observation.

Implementation: matches exactly (`select_atlas_anchors`, §9). Its dominant cost, $O(n_a\, n \log n)$, is the single largest unresolved complexity lever in the whole pipeline (§9 Implementation).

1.23.4 Stitching/topology

This is currently the least mathematically settled component.

The developing principle is:

$$\boxed{\text{chart overlap proposes connectivity; data-supported local geometry decides connectivity.}}$$

Residual/noise structure, tangent compatibility and observed support around a junction provide the evidence.

Implementation status (2026-08-05): this remains true, but for a reason this document didn't originally anticipate. "Chart overlap proposes connectivity" is implemented (Phase VIII's candidate graph). "Data-supported local geometry decides connectivity" is **not** implemented as residual/noise structure or tangent compatibility (§13-15, both open) — instead, connectivity is decided by the anchor MST (§17): Tier-1 edges from

Step 10's own primary/secondary anchor ranking (itself driven by the scalar inverse-variance weight, §16), Tier-2 edges from raw geometric sphere overlap. This is a real, working, extensively-debugged (§17's three-attempts history) mechanism — not a placeholder — but it decides connectivity by a different kind of evidence (anchor-graph topology + raw distance) than "residual/noise structure and tangent compatibility" as originally envisioned. Whether to still build the originally-envisioned noise-tube/Mahalanobis/tangent-angle gate (§13-15) *in addition to* the working MST mechanism — e.g. as a stricter Tier-0 that could *reject* an edge the MST would otherwise accept — is the central open question left in this document.

1.23.5 Iterative refinement

Refinement is also algorithmic rather than part of the first-order tangent approximation. Its purpose is removal of finite-sample and stitching artifacts without destroying the recovered geometry.

Implementation: the purpose matches, but the mechanism is a full atlas rebuild-and-restitch (§18), not a separate smoothing kernel F — and its stopping rule is a bare displacement check, not the quality-weighted objective §19 describes (open experiment). This iterative structure is, per this session's discussion, understood to be load-bearing: without it, earlier non-iterative versions of the pipeline produced multiple parallel/nearby spurious manifolds instead of collapsing to one.

1.24 Principal limitations that must remain explicit

LoManLe does **not** escape the curse of dimensionality.

Its most important expected failure modes are:

High ambient dimension D. Euclidean nearest-neighbor relationships may deteriorate, and accumulated noise across many normal dimensions can obscure tangent structure.

> Observed instance: `misc/smoothing_experiments.md` §18.2 and §18.6 document this concretely, not just abstractly — a k-d tree range/kNN query degrades toward a full $O(n)$ scan once the query radius/count covers a large fraction of the dataset, regardless of D per se, but the effect is the practical manifestation of this limitation. No alternative high-D index has been implemented or evaluated (§4 Implementation).

High intrinsic dimension d. Reliable estimation of a d-dimensional tangent space requires increasingly many local observations.

Sparse sampling. No local algorithm can recover geometry that is unsupported by observations.

High curvature. A neighborhood large enough for stable variance estimation may already be too large for a first-order tangent approximation.

Nearby folds/branches. k-nearest-neighbor neighborhoods can combine observations that are close in ambient Euclidean distance but far apart geodesically.

Junctions. A single tangent space may be undefined or misleading at a true bifurcation/multifurcation.

> Observed instance: this is exactly why §6's growth rule reacts to *instability*, not just an insufficient spectral gap — a neighborhood that has silently crossed into a different branch produces an oversized sphere, an anchor covering more than one branch, and downstream spurious intersections (§6 Implementation, "why this matters downstream").

Boundaries. Local covariance becomes asymmetric near manifold boundaries and requires explicit recognition.

Variable density. Neighborhood size and reliability must adapt locally.

These are not merely implementation issues; they define the regime in which the method must be experimentally characterized.

1.25 Compact definition

LoManLe can therefore be defined as:

> **An adaptive local manifold estimator that reconstructs a noisy low-dimensional manifold embedded in multivariate space by finding the smallest neighborhoods supporting stable**

local tangent-space estimates, representing those estimates as overlapping affine charts, orthogonally projecting observations onto the charts, inferring global topology from data-supported chart relationships, stitching corresponding projections, and optionally refining the resulting manifold while controlling reconstruction quality and roughness.

Or mathematically, its core operation is simply

$$\boxed{\mathbf{x} \longmapsto \boldsymbol{\mu}_a + U_a U_a^\top (\mathbf{x} - \boldsymbol{\mu}_a)}$$

where both the appropriate local scale and the local tangent basis U_a are learned from the noisy point cloud.

Everything around that operation answers three questions:

At what scale is this local approximation trustworthy?
Which local approximations belong to the same manifold?
How should those approximations be assembled globally?

Those are, at present, the defining problems LoManLe is trying to solve.

API status (`misc/smoothing_experiments.md`, "Status: experimental, and the API reflects that"): `lomanle_compute_alloc` currently exposes many intermediate/diagnostic arrays beyond what a finished "just give me the skeleton" interface would — intentional while the open experiments listed throughout this document (§7, §13, §14, §15, §19, and the $d > 1$ mesh in §17) are still being explored, and flagged there as needing to be trimmed once the design settles.

1.26 Open-experiments index

A single place to find every "not yet implemented / actively being explored" item this document flags, for anyone picking this up next:

#	Open experiment	Where
1	Local intrinsic dimension inference (d_i per chart)	§7
2	Bandwidth-field smoothing for σ_i	§8
3	Event-driven / priority-queue restructuring of Step 5's anchor selection (remove the $O(n_a)$ full-candidate-rescan factor)	§9 — largest remaining complexity lever
4	Anisotropic per-anchor noise covariance $\Sigma_{\perp,a}$ (currently only a scalar `normal_error`)	§13 — live design discussion, 2026-08-04
5	Noise-tube / Mahalanobis stitch-decision gate ($D_{ab}^2 \le \tau_{\text{noise}}$)	§14 — "Potential Stitching Experiment" in the lab book
6	Principal-angle tangent-compatibility test between distinct anchors (continuation/furcation/near-miss)	§15
7	$d > 1$ mesh/simplicial topology (currently backbone is 1-D by construction regardless of `manifold_dim`)	§17
8	Roughness/coverage/RMSE weighted stopping objective for `lomanle_compute`'s own outer loop (currently `max_disp`-only)	§19

#	Open experiment	Where
9	High-D nearest-neighbor index alternative (k-d tree deterioration)	§4, §22
10	Large-$k_{\min}$ `kd_knn_query` cost in Steps 1-3 (no known low-risk fix)	§6
11	Bifurcation validation gaps: T-shapes, multiple simultaneous junctions, uneven-density bifurcations	§15
12	Trim `lomanle_compute_alloc`'s wide diagnostic API once the above settle	§2, §23

Items 3 and 7 are, per the lab book's own framing, the highest-value next steps: 3 is the largest remaining performance lever, and 7 is the largest remaining capability gap ($d = 1$ curves work end-to-end; surfaces do not yet get an explicit topology at all).